In [2]:

```
#for data operations and cleaning we import pandas
import pandas as pd

#we import z score from scipy.stats to showcase skewed data in all attributes
from scipy.stats import zscore

#we import seaborn as sns to implement a display technique for visual
#representation of data for human perception
import seaborn as sns

#we import matplotlib.pyplot as plt to implement a display
#technique for visual representation of data for human perception
import matplotlib.pyplot as plt

#we require label encoder which we import from sklearn.preprocessng
from sklearn.preprocessing import LabelEncoder
```

In [3]:

```
path1 = "employee_data.csv"
path2 = "Iris.csv"

#Read employee_data.csv: load first dataset into df1.
df1 = pd.read_csv(path1)

#Read Iris.csv: load second dataset into df2.
df2 = pd.read_csv(path2)
```

In [4]:

```
#check first few rows to see structure.
df1.head
```

Out[4]:
```
<bound method NDFrame.head of      Index  Employee_ID  Age Department  Is_Manager   Join_Date  Experience  \
0        0         8794   60    Finance       False  1994-03-24          30
1        1         8222   28      Sales        True  1979-12-08          13
2        2         7643   30  Marketing       False  1998-09-25          11
3        3         2737   38         HR        True  2011-11-23          20
4        4         8557   61         HR        True  1976-10-08          31
..     ...          ...  ...        ...         ...         ...         ...
495    495         8426   32  Marketing        True  2013-12-04          16
496    496         9810   26         IT        True  2022-05-02          15
497    497         5396   34    Finance       False  2025-04-22          16
498    498         7546   60         HR       False  1970-01-13          28
499    499         7744   29    Finance        True  1988-06-21          15

     Salary
0    238668
1    105995
2     96120
3    153525
4    249192
..      ...
495  126088
496  118094
497  131208
498  224502
499  114758

[500 rows x 8 columns]>
```

In [5]:
```
#df1.describe: show summary stats for numeric columns in df1.
df1.describe()
```

Out[5]:

	Index	Employee_ID	Age	Experience	Salary
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	249.500000	5440.010000	43.562000	21.316000	170634.372000
std	144.481833	2609.366682	11.857798	6.296621	50611.711724
min	0.000000	1002.000000	22.000000	7.000000	43177.000000
25%	124.750000	3219.000000	34.000000	16.000000	130346.000000
50%	249.500000	5273.500000	44.000000	22.000000	172752.500000
75%	374.250000	7658.000000	52.000000	26.000000	208607.500000
max	499.000000	9981.000000	64.000000	36.000000	293410.000000

In [6]:

```
#df2.describe: show summary stats for numeric columns in df2.
df2.describe()
```

Out[6]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [7]:
```
#Define bins and labels: create boundaries for age ranges.
bins = [20, 34, 50, float('inf')]
labels = [1, 2, 3]

#pd.cut on Age: classify each employee into an age_group.
df1['age_group'] = pd.cut(df1['Age'], bins=bins, labels=labels, right=True, include_lowest=True)
```

In [8]:
```
##check first few rows to see structure.
df1.head
```

Out[8]:
```
<bound method NDFrame.head of      Index  Employee_ID  Age Department  Is_Manager   Join_Date  Experience  \
0        0         8794   60    Finance       False  1994-03-24          30
1        1         8222   28      Sales        True  1979-12-08          13
2        2         7643   30  Marketing       False  1998-09-25          11
3        3         2737   38         HR        True  2011-11-23          20
4        4         8557   61         HR        True  1976-10-08          31
..     ...          ...  ...        ...         ...         ...         ...
495    495         8426   32  Marketing        True  2013-12-04          16
496    496         9810   26         IT        True  2022-05-02          15
497    497         5396   34    Finance       False  2025-04-22          16
498    498         7546   60         HR       False  1970-01-13          28
499    499         7744   29    Finance        True  1988-06-21          15

     Salary age_group
0    238668         3
1    105995         1
2     96120         1
3    153525         2
4    249192         3
..      ...       ...
495  126088         1
496  118094         1
497  131208         1
498  224502         3
499  114758         1

[500 rows x 9 columns]>
```

In [9]:
```
#groupby age_group on Salary: compute grouped salary stats.
grouped = df1.groupby('age_group')['Salary'].agg(['mean', 'median', 'min', 'max', 'std'])

#print(grouped): display results of grouped stats.
print(grouped)
```

```
                   mean    median     min     max           std
age_group
1          108274.395522  111729.0   43177  152895  21468.340681
2          169144.043269  170646.0  103682  226499  24763.174831
3          225483.898734  225719.0  148735  293410  24349.142135
/tmp/ipykernel_4274/1294133543.py:1: FutureWarning: The default of observed=False is deprecated and will be change
d to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt
the future default and silence this warning.
  grouped = df1.groupby('age_group')['Salary'].agg(['mean', 'median', 'min', 'max', 'std'])
```

In [11]:
```
#df2.describe: confirm Iris stats again after load.
df2.describe()
```

Out[11]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [12]:
```
#df2.head: preview Iris dataset rows.
df2.head()
```

Out[12]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

In [15]:

```
#mapping dict: assign numeric codes to Iris species names.
mapping = {
    'Iris-setosa': 1,
    'Iris-versicolor': 2,
    'Iris-virginica': 3
}

#map Species to Species_code: apply mapping to df2.
df2['Species_code'] = df2['Species'].map(mapping)
```

In [16]:

```
#df2.head: verify encoded species column.
df2.head()
```

Out[16]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Species_code
0	1	5.1	3.5	1.4	0.2	Iris-setosa	1
1	2	4.9	3.0	1.4	0.2	Iris-setosa	1
2	3	4.7	3.2	1.3	0.2	Iris-setosa	1
3	4	4.6	3.1	1.5	0.2	Iris-setosa	1
4	5	5.0	3.6	1.4	0.2	Iris-setosa	1

In [19]:

```
#groupby Species_code with agg: compute count and basic stats per species.
stats = df2.groupby('Species_code').agg(
    count=('Species_code', 'count'),
    mean=('Species_code', 'mean'),
    median=('Species_code', 'median'),
    std=('Species_code', 'std'),
    mode=('Species_code', lambda x: x.mode().iloc[0])
)
print(stats)
```

```
              count  mean  median  std  mode
Species_code
1                50   1.0     1.0  0.0     1
2                50   2.0     2.0  0.0     2
3                50   3.0     3.0  0.0     3
```